INFORMATIONSBLATT

IB01 – Arrays: viele Werte unter einem Namen

Dynamische Webseiten mit PHP · LS06 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

> 🎯 **Darum geht es**
>
> Sie speichern Listen von Werten in einem Array, greifen über den Schlüssel auf einzelne Werte zu, verändern und erweitern Arrays und nutzen die wichtigsten Array-Funktionen.

1 | Was ist ein Array?

Eine Variable speichert *einen* Wert. Ein **Array** speichert beliebig viele – wie ein Karteikasten mit nummerierten oder beschrifteten Fächern. Jedes Fach hat einen **Schlüssel** (engl. *key*) und einen **Wert**.

Indiziertes Array `$sorten = ["Vanille", "Mango", "Kokos"];`

0	1	2
"Vanille"	"Mango"	"Kokos"

Schlüssel (Index): automatisch ab 0

Wert

`$sorten[1] → "Mango"`

Assoziatives Array `$preise = ["Waffel" => 1.70, "Becher" => 7.30];`

"Waffel"	"Becher"
1.70	7.30

Schlüssel: selbst gewählte Namen

Wert

`$preise["Becher"] → 7.30`

Abbildung 1: Zwei Arten von Arrays: nummerierte und benannte Schlüssel.

2 | Indizierte Arrays

Man legt ein Array mit eckigen Klammern an. Die Schlüssel vergibt PHP automatisch – beginnend bei **0**.

Anlegen, lesen, ändern, erweitern beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Arrays</title></head>
<body>

<?php
  $sorten = ["Vanille", "Mango", "Kokos"];

  echo "<p>Erste Sorte: " . $sorten[0] . "</p>";
  echo "<p>Anzahl: " . count($sorten) . "</p>";

  $sorten[1] = "Maracuja";      // Wert ersetzen
  $sorten[]  = "Pistazie";      // hinten anhängen

  echo "<pre>";
  print_r($sorten);
  echo "</pre>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

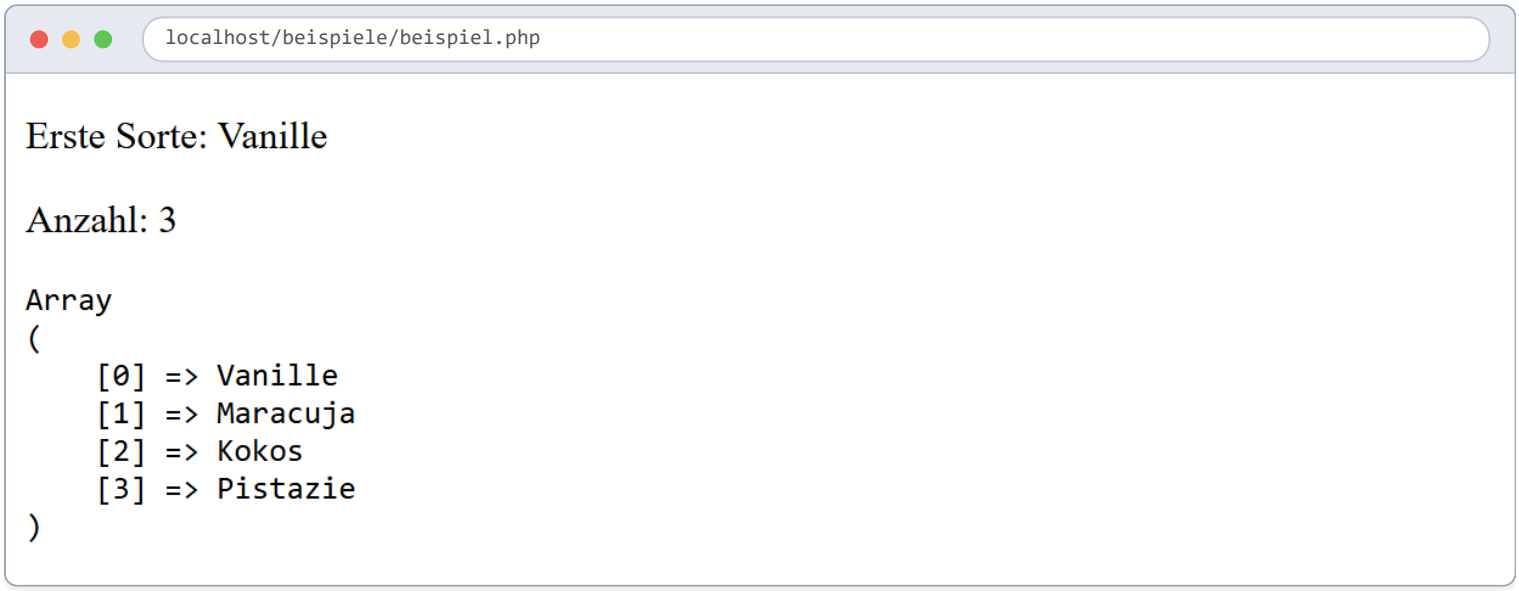


Abbildung 2: Zugriff, Ersetzen und Anhängen; print_r zeigt den ganzen Inhalt.

⚠️ DER ERSTE IST NUMMER 0

Bei vier Einträgen gibt es die Schlüssel 0, 1, 2 und 3. `$sorten[4]` existiert nicht – PHP meldet *Undefined array key 4*. Der letzte Eintrag ist immer `$sorten[count($sorten) - 1]`.

3 | Nützliche Array-Funktionen

Funktion	Aufgabe	Beispiel
`count($a)`	Anzahl der Einträge	4
`in_array($wert, $a)`	Ist der Wert enthalten? (true/false)	`in_array("Mango", $sorten)`
`sort($a)`	Werte aufsteigend sortieren (verändert das Array)	A → Z, 1 → 9
`implode(", ", $a)`	alle Werte zu einem Text verbinden	„Apfel, Erdbeere, …“
`array_sum($a)`	Summe aller Zahlen	`array_sum([2, 3, 5])` → 10
`print_r($a)`	Inhalt anzeigen (zum Testen)	

Suchen, sortieren, verbinden beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Array-Funktionen</title></head>
<body>

<?php
  $sorten = ["Zitrone", "Apfel", "Mango", "Erdbeere"];

  if (in_array("Mango", $sorten)) {
    echo "<p>Mango ist heute da.</p>";
  }

  sort($sorten);                              // alphabetisch sortieren
  echo "<p>" . implode(", ", $sorten) . "</p>"; // zu einem Text verbinden
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

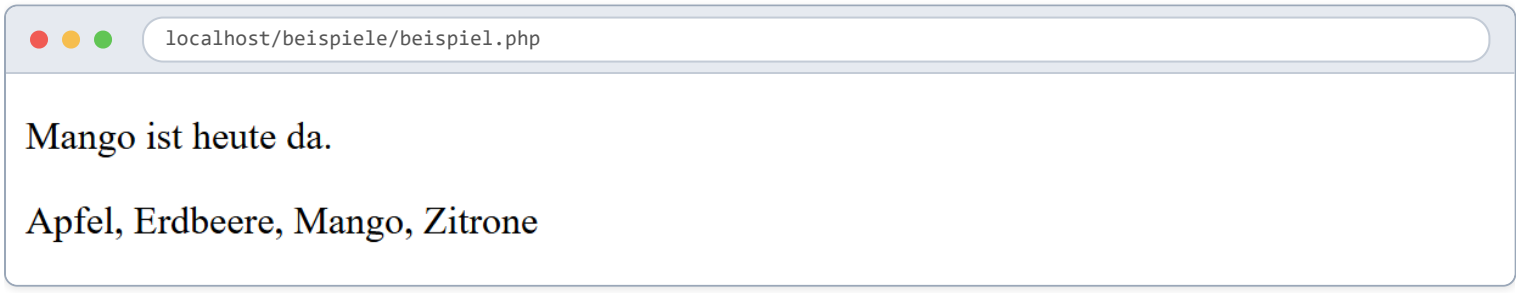


Abbildung 3: in_array() sucht, sort() ordnet, implode() verbindet.

4 | Assoziative Arrays

Statt Nummern kann man eigene Namen als Schlüssel vergeben. Der Pfeil `=>` ordnet jedem Schlüssel seinen Wert zu. So werden Arrays selbsterklärend: `$preise["Becher"]` versteht jeder.

Schlüssel mit Namen beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>assoziativ</title></head>
<body>

<?php
  $preise = [
    "Waffel"      => 1.70,
    "Becher"      => 7.30,
    "Milchshake"  => 4.20,
  ];

  echo "<p>Ein Milchshake kostet "
       . number_format($preise["Milchshake"], 2, ",", ".") . " €.</p>";

  $preise["Eiskaffee"] = 4.90;     // neuer Eintrag
  echo "<p>Einträge: " . count($preise) . "</p>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS



Abbildung 4: Zugriff über den Namen; neue Einträge entstehen durch Zuweisung.

DAS KENNEN SIE SCHON

`$_POST` und `$_GET` sind assoziative Arrays: `$_POST["name"]` ist der Wert unter dem Schlüssel „name“. Deshalb zeigte `print_r($_POST)` in LS03 dieselbe Darstellung.